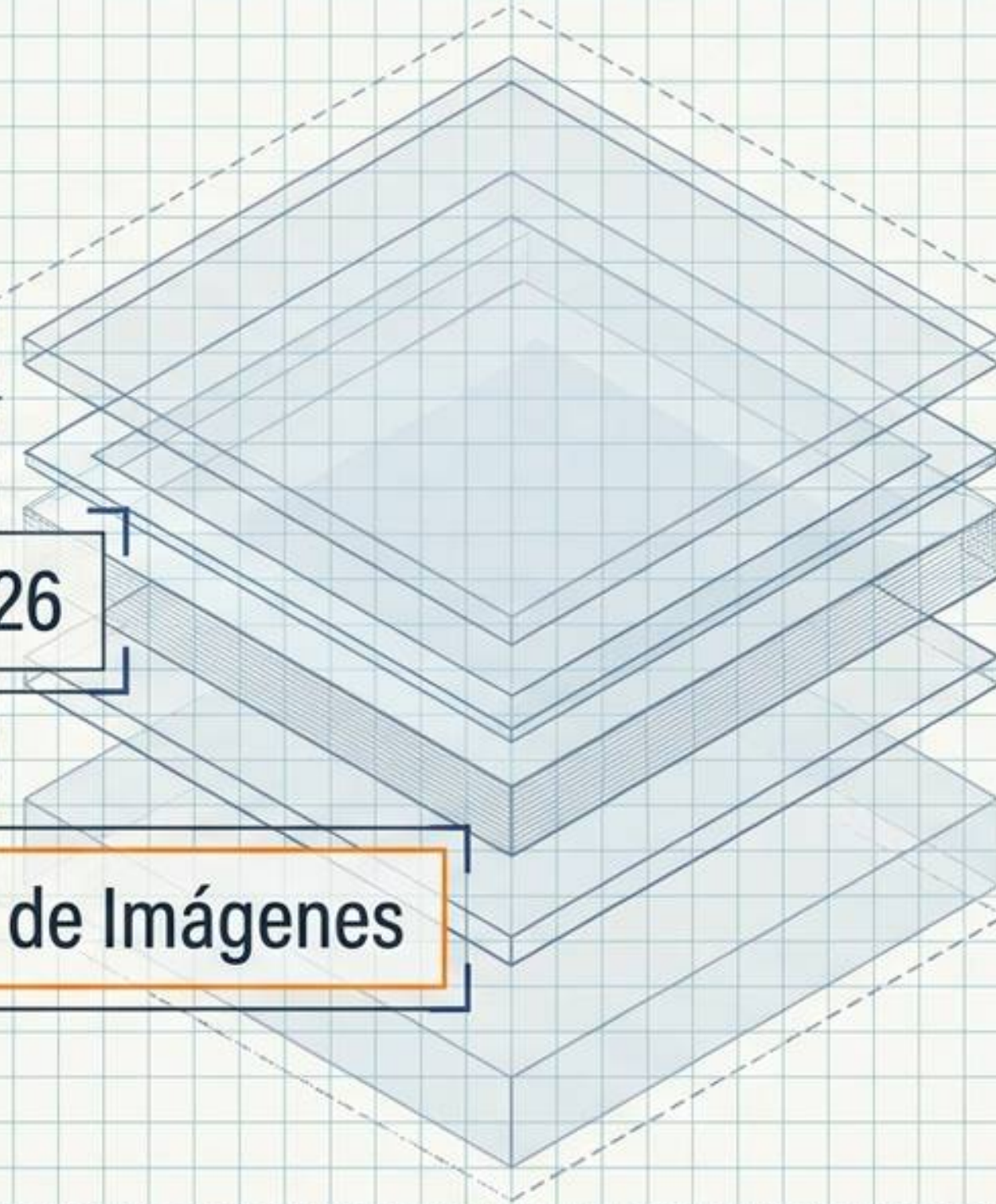


Fundamentos de Deep Learning

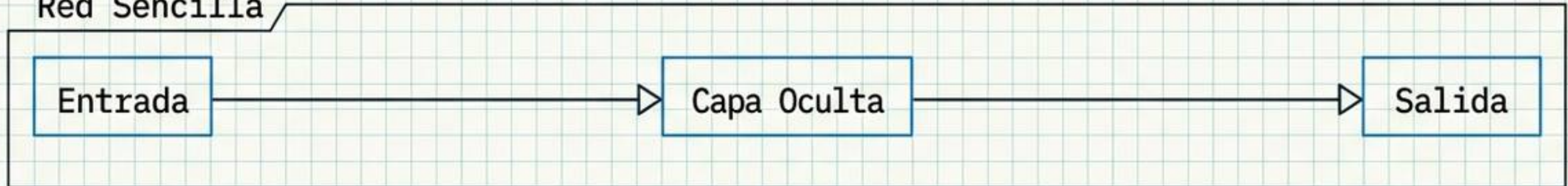
Semana 9 — Machine Learning 2026

 Proyecto Integrador: Clasificación de Imágenes

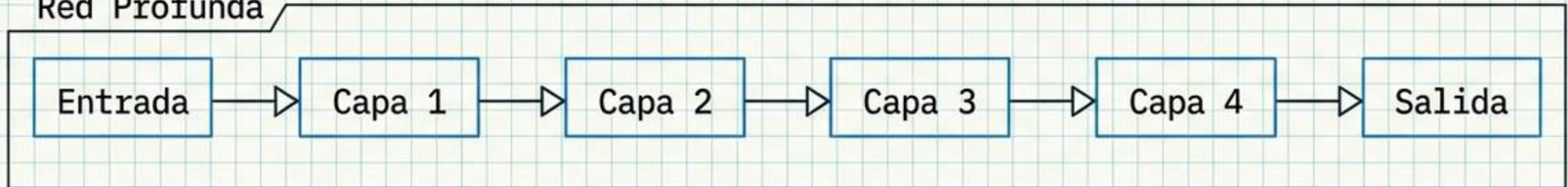


La profundidad permite aprender representaciones progresivamente complejas

Red Sencilla



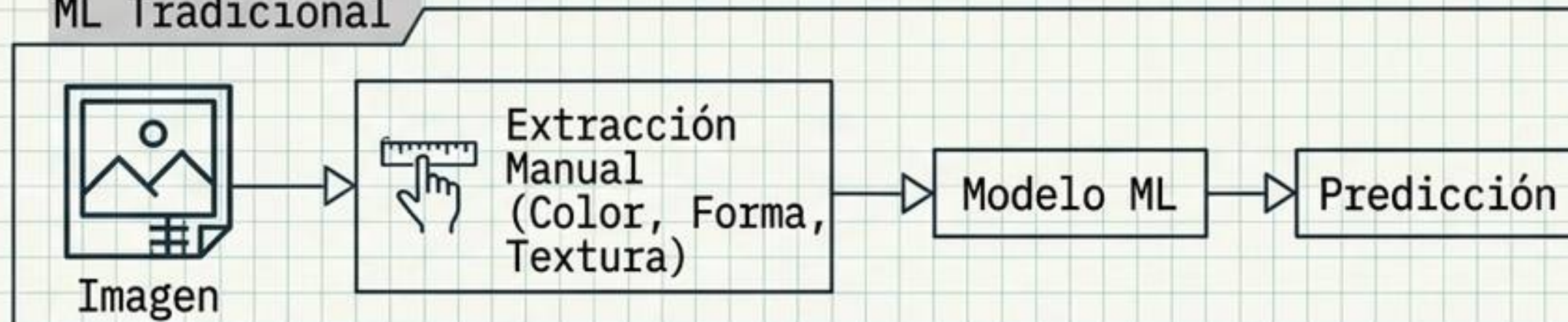
Red Profunda



Deep Learning mantiene los mismos principios básicos (pesos, sesgos, backpropagation), pero utiliza **múltiples capas**. Cada capa transforma la representación recibida desde la anterior.

Aprendizaje Jerárquico vs. Feature Engineering Manual

ML Tradicional



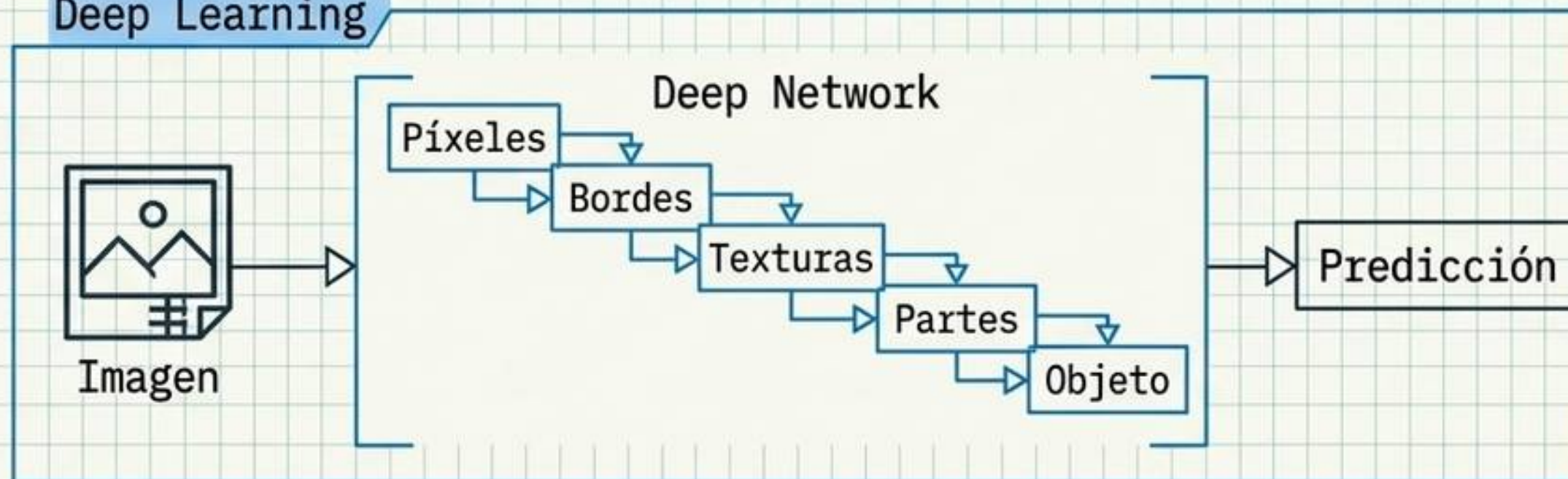
El reto de las imágenes:

Una imagen de $224 \times 224 \times 3$ contiene 150.528 valores.

Definir reglas manuales para distintos ángulos, iluminaciones y escalas es inviable.

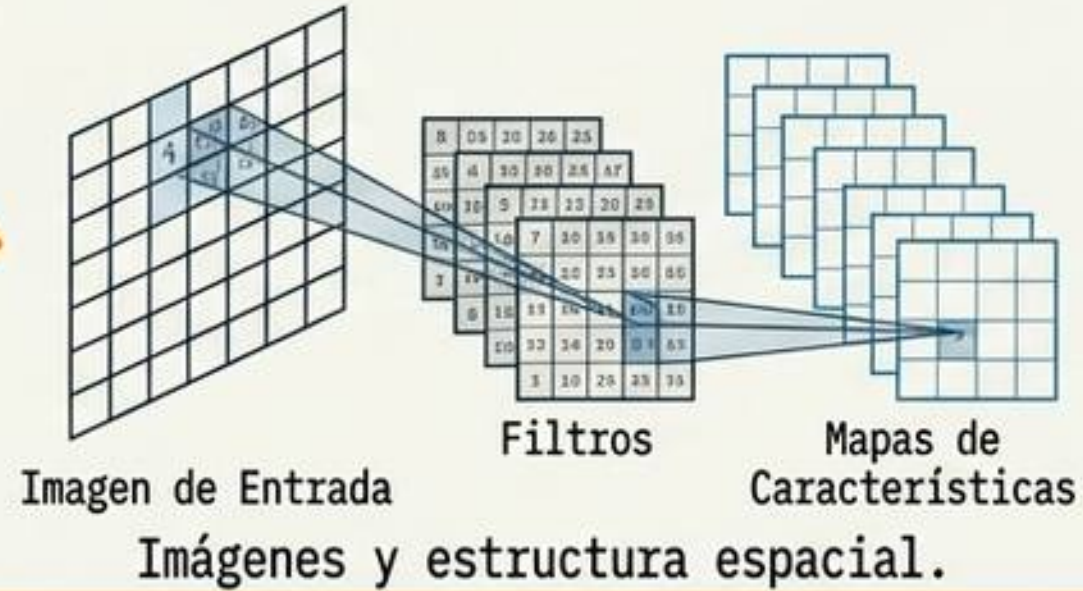
DL automatiza esta extracción.

Deep Learning



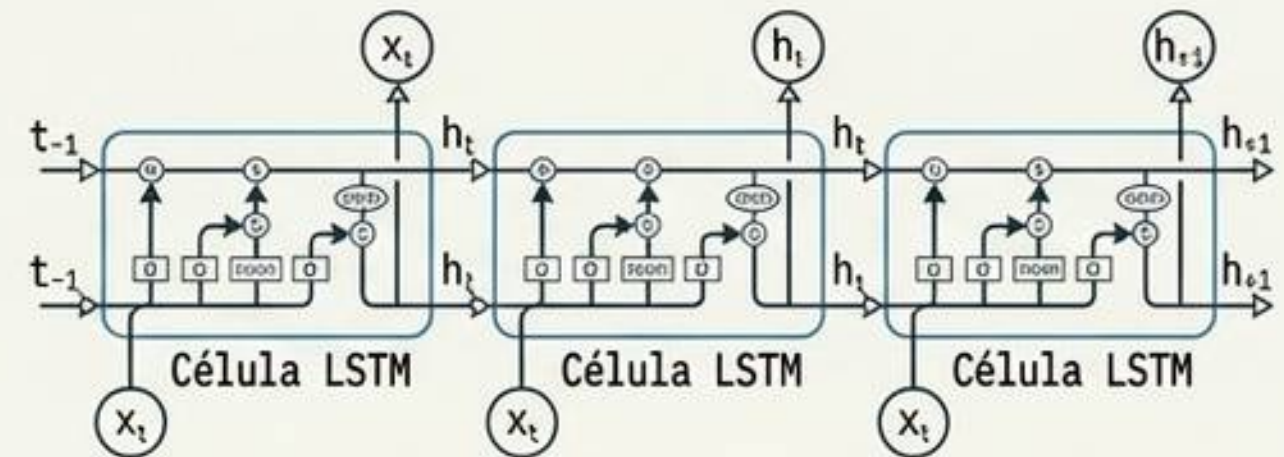
Distintas arquitecturas estructurales para distintos problemas

1. CNN (Redes Convolucionales)



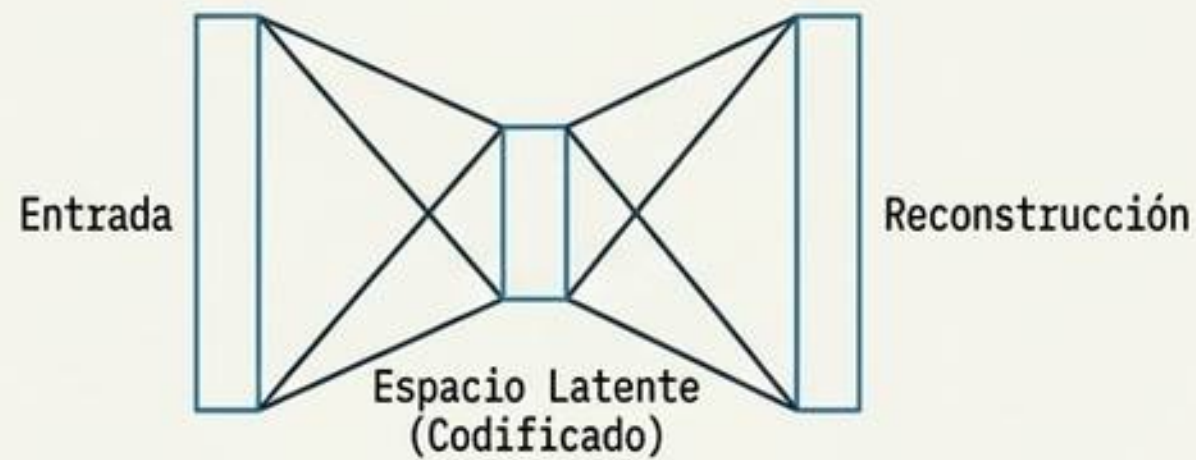
Núcleo de
nuestro Proyecto
Integrador

2. LSTM



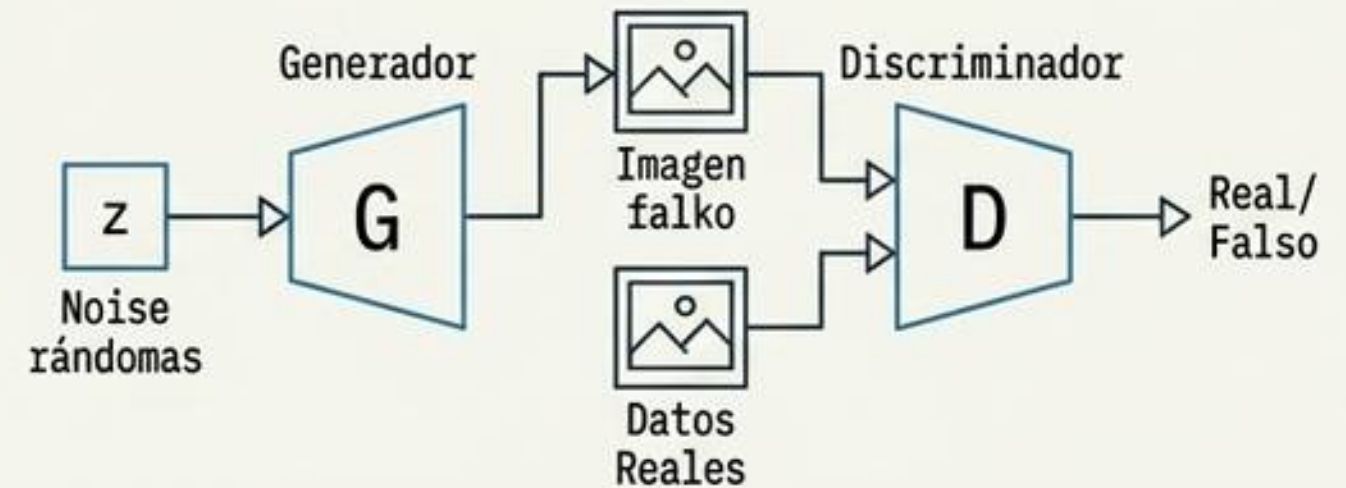
Información secuencial y series de tiempo.

3. Autoencoders



Representaciones comprimidas de datos.

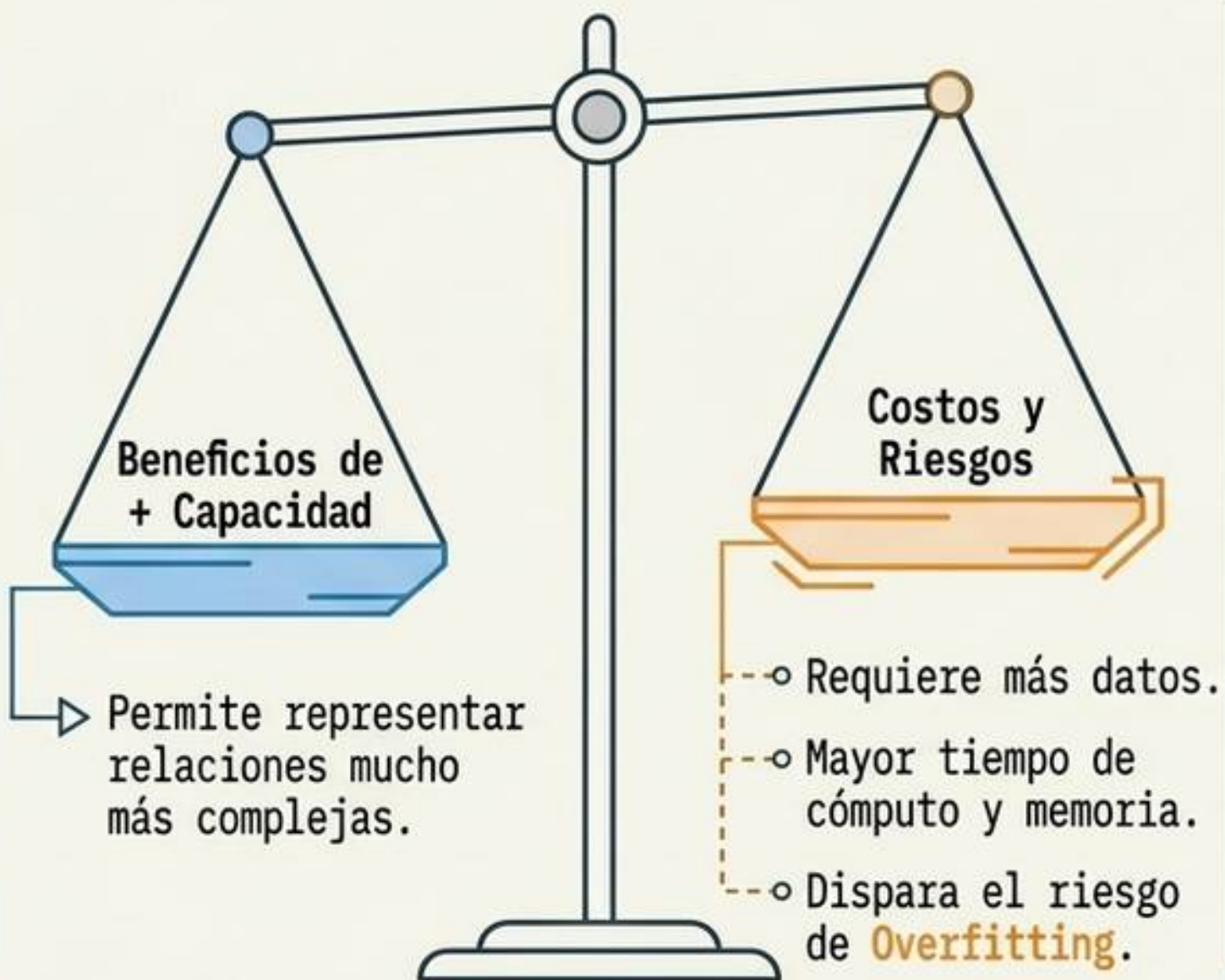
4. GAN



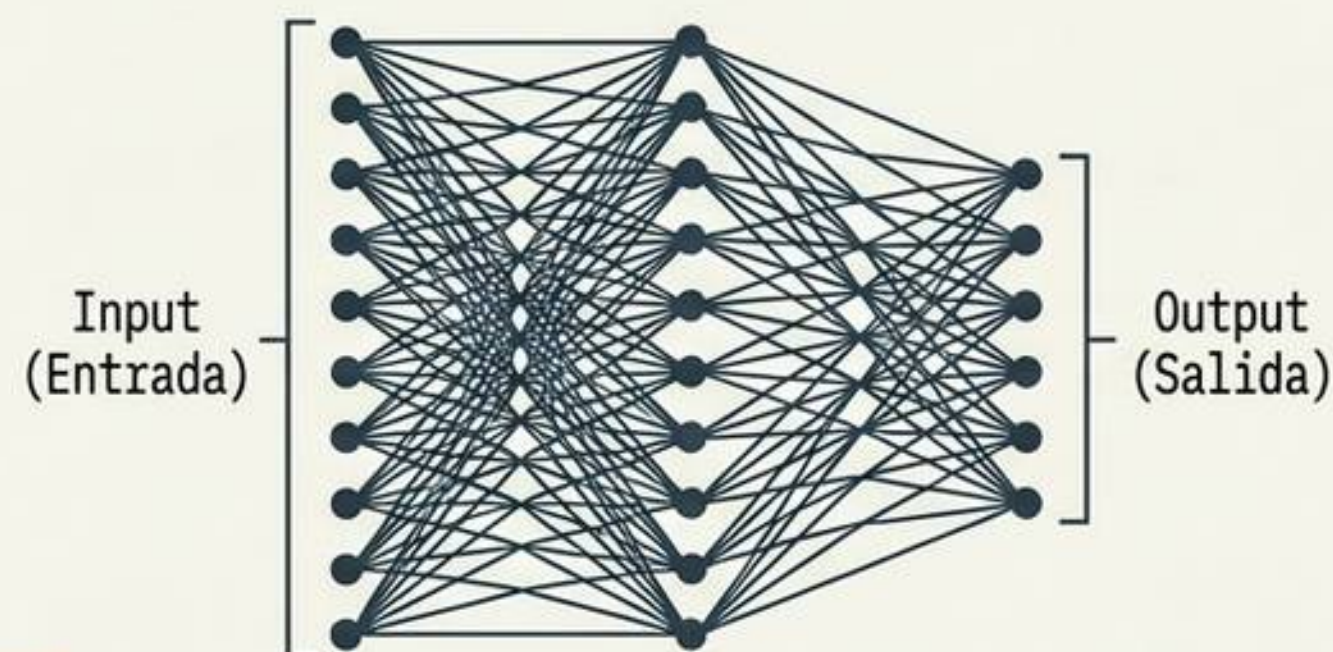
Redes adversarias para generación de nuevas muestras.

Capas Densas, Capacidad del Modelo y el Costo de la Profundidad

El Compromiso (The Trade-off)



Panel Técnico - Esquema Capa Densa



Ejemplo de Parámetros en Capa Densa

Entrada: 100 valores → Salida: 50 neuronas

$(100 \times 50 \text{ pesos}) + 50 \text{ sesgos} = 5.050 \text{ parámetros entrenables}$

Modelos modernos pueden tener millones de parámetros.

Diferenciando los componentes: Parámetros vs. Hiperparámetros

Parámetros

Origen

Los aprende y modifica la red automáticamente durante el entrenamiento iterativo.

Ejemplos

Pesos, Sesgos

Hiperparámetros

Origen

Decisiones de diseño configuradas por el desarrollador de machine learning antes o durante el proceso.

Ejemplos

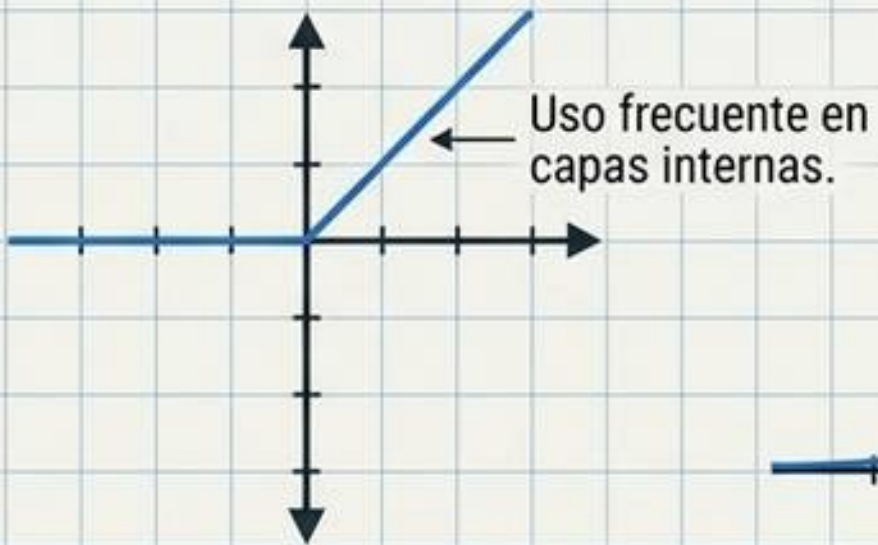
Número de capas, Neuronas por capa, Learning rate, Batch size, Épocas, Optimizador, Funciones de activación

→ Frontera: Arquitectura Aprendida vs. Diseño Humano

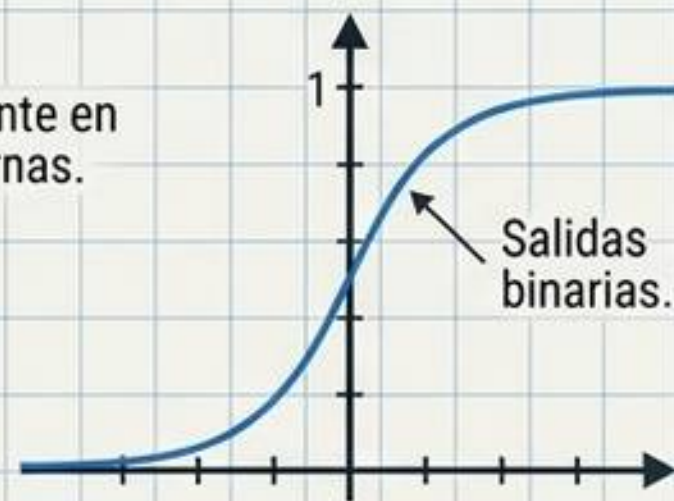
Funciones de Activación y el Cálculo de Pérdida (Loss)

Activaciones (La No-linealidad)

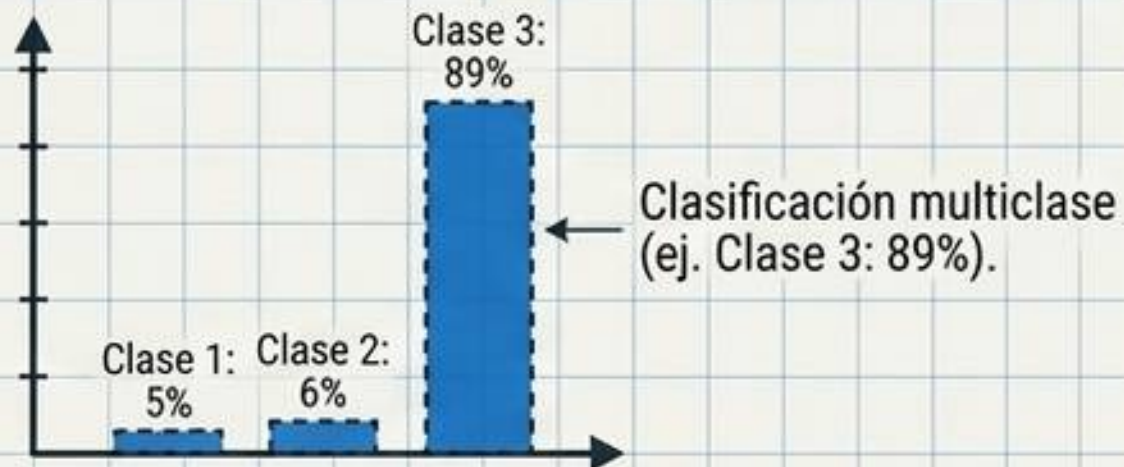
ReLU



Sigmoid



Softmax



Loss (El Error)

[Predicción del modelo: Plástico]

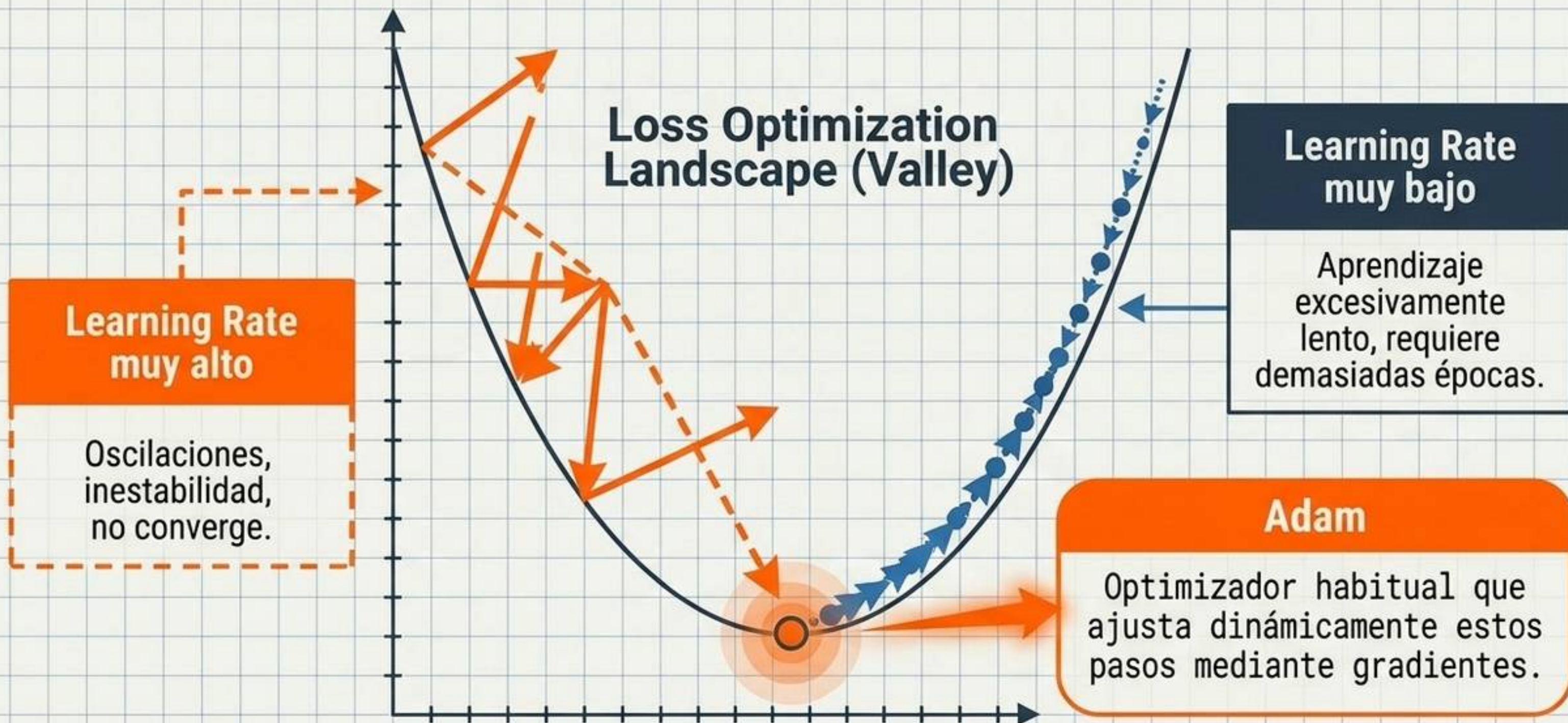
[Etiqueta Real del dataset: Vidrio]



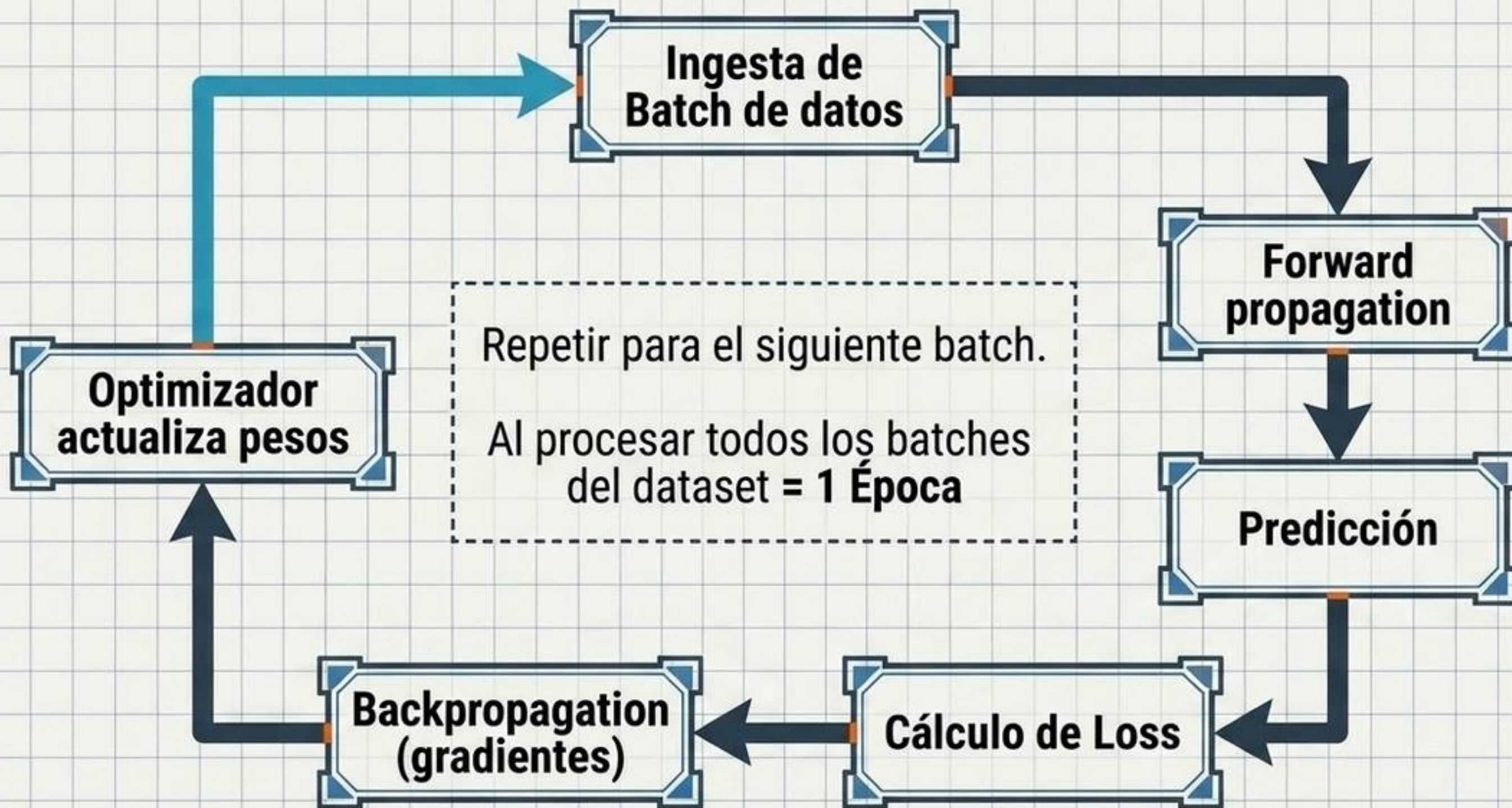
Alta señal de error

Función habitual multiclase:
Categorical crossentropy

Optimización: Guiando la actualización de los parámetros



El ciclo continuo de entrenamiento por Épocas y Batches



Construcción de la arquitectura en código (Keras)

```
modelo = Sequential()  
modelo.add(Dense(128, activation='relu'))  
modelo.add(Dense(64, activation='relu'))  
modelo.add(Dense(4, activation='softmax'))
```

```
modelo.summary()
```

Estructura progresiva
capa por capa.

Salida configurada para
4 clases distintas.

summary

Herramienta vital para
auditar la forma de la
salida y la cantidad de
parámetros entrenables.

Automatización del ciclo: Compile, Fit y Evaluate

1. Configurar (compile)

```
optimizer='adam', loss='categorical_crossentropy'
```

Define el optimizador y la función de pérdida matemática.

2. Entrenar (fit)

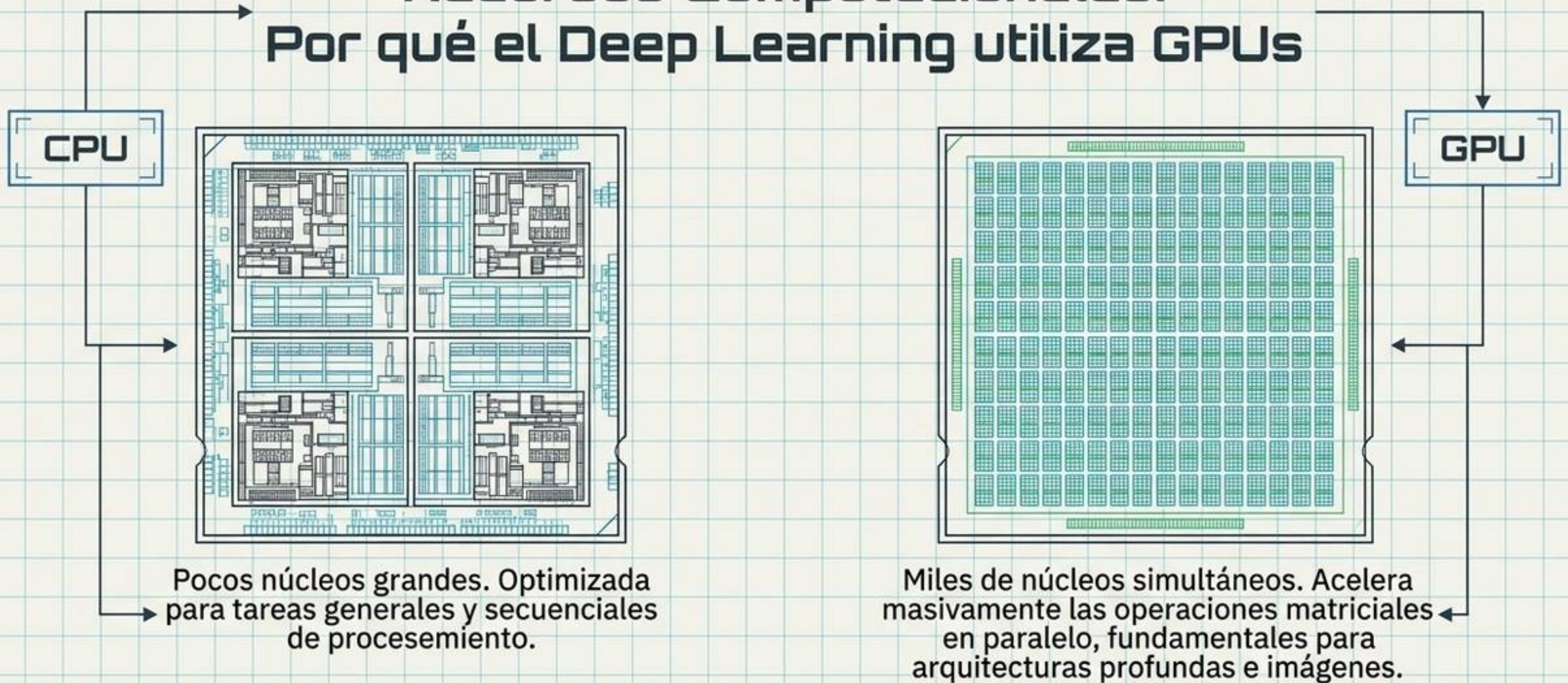
```
epochs=50, batch_size=32,  
validation_data=(X_val, y_val)
```

Ejecuta el ciclo. Usa Training para aprender y Validation para seleccionar/monitorear sin aprender de ellos.

3. Medir (evaluate) (X_test, y_test)

Evaluación final e imparcial, ciega al modelo, usando **Test**.

Recursos Computacionales: Por qué el Deep Learning utiliza GPUs



Dependencia de recursos (RAM, Storage) crece geométricamente según el tamaño de la imagen, batch size y dataset total.

El ciclo iterativo requiere experimentación controlada

lab notebook

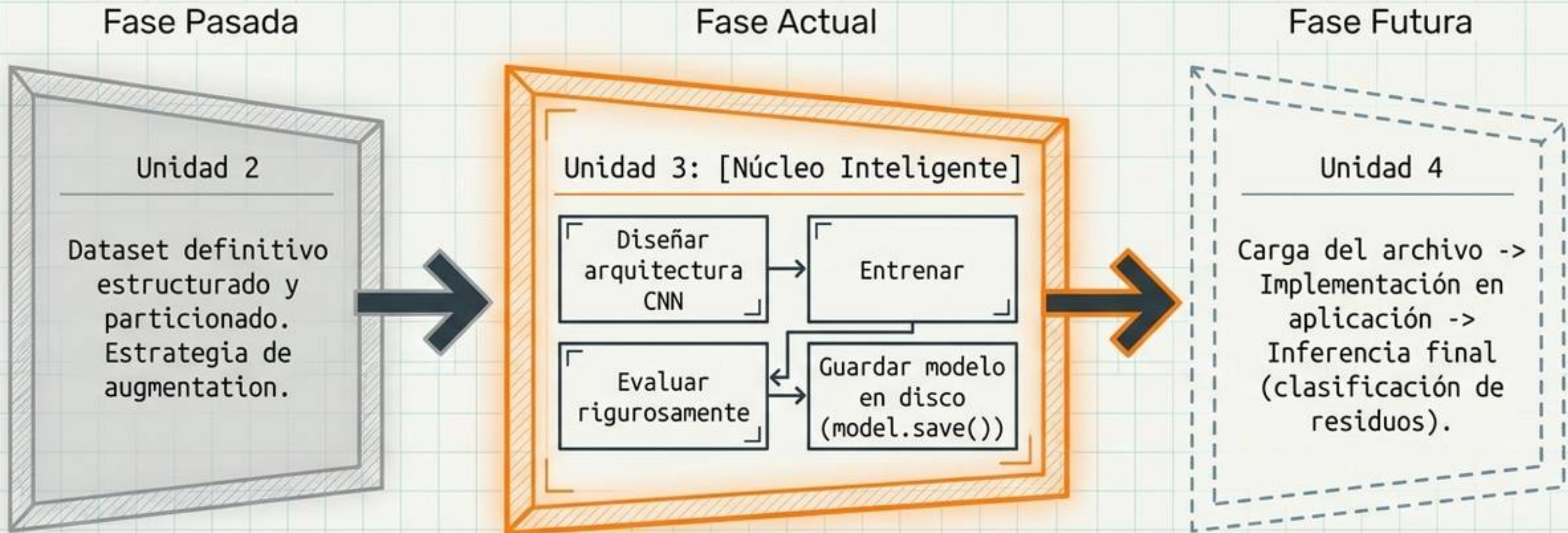
Modelo	Capas	Learning Rate	Val Accuracy
Model A	2	0.001	81%
Model B	3	0.001	87%
Model C	5	0.010	72%

Regla Crítica de Experimentación

Para que una comparación sea válida y reproducible, se deben aislar las variables. Se debe mantener constante: El mismo dataset, misma partición, misma estrategia de Data Augmentation y mismas métricas.

Si modificas cuatro variables simultáneamente, no sabrás qué produjo la mejora.

Del Modelo al Producto: El Proyecto Integrador



Síntesis y Puente hacia la Semana 10

- ✓ **Profundidad** = transformaciones abstractas de datos.
- ✓ Más capacidad = riesgo de **Overfitting** y mayor costo computacional.
- ✓ **Parámetros** (aprendidos iterativamente) vs. **Hiperparámetros** (diseñados manualmente).
- ✓ **Entrenamiento** = un ciclo matemático automatizable (Keras) que requiere experimentación estricta y controlada.

Pregunta: ¿Cómo puede una red aprender **automáticamente** dónde hay **bordes**, **texturas** y **formas** en una matriz incomprensible de 150.528 píxeles?

Próxima Parada: Semana 10 — **Redes Neuronales Convolucionales (CNN)** y la extracción **espacial** de características.